

LLM-Agents and more

Yandex School of Data Analysis,
Iaroslav Khripkov

TLDR; What an LLM-Agent is

LLM-Agent:

1. LLM
2. Prompt
3. Tools | None
4. Reasoning | None

Lecture plan

1. Motivation
2. What is an AI Agent?
3. LLM-Agents Predecessors
4. Tools
5. Memory
6. Multi-step Agents
7. Multi-Agent Systems
8. Agentic RAG
9. Training
10. Eval
11. Stack
12. Applications

Motivation

Large Language Models are powerful, but they are like brilliant minds locked in a room. We'll explore their inherent limitations and how the "agent" paradigm unlocks their potential.

Key Deficiencies of Standard LLMs:

- **Lack of Action & Tool Use:** Cannot execute code, perform calculations, or interact with the outside world. They can only generate text.
- **Static Knowledge:** Their knowledge is frozen at the time of their last training, leading to outdated or "*hallucinated*" information.
- **Limited Context & Memory:** They lack true long-term memory beyond the context window of a single conversation.
- **Passive Interaction Model:** They cannot self-initiate tasks, seek feedback, or operate autonomously over long periods. They must be prompted for every turn.
- **Interfacing Challenges:** No native ability to access external APIs, databases, or file systems.
- **Need for Oversight:** Require human guidance for complex, multi-step tasks and clearance for critical actions.

What is an AI Agent?

AI agents are software systems that use AI to pursue goals and complete tasks on behalf of users. They show reasoning, planning, and memory and have a level of autonomy to make decisions, learn, and adapt.[1]

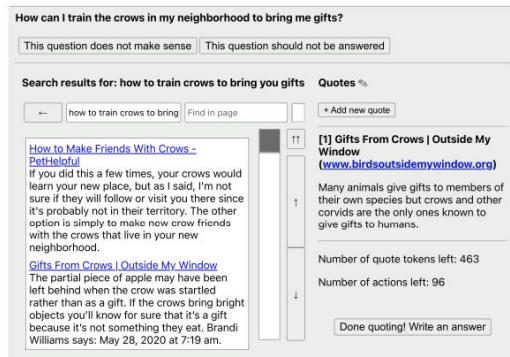
Agency Level	Description	Short name	Example Code
☆☆☆	LLM output has no impact on program flow	Simple processor	<code>process_llm_output(llm_response)</code>
★☆☆	LLM output controls an if/else switch	Router	<code>if llm_decision(): path_a() else: path_b()</code>
★★☆	LLM output controls function execution	Tool call	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
★★★	LLM output controls iteration and program continuation	Multi-step Agent	<code>while llm_should_continue(): execute_next_step()</code>
★★★★	One agentic workflow can start another agentic workflow	Multi-Agent	<code>if llm_trigger(): execute_agent()</code>
★★★★	LLM acts in code, can define its own tools / start other agents	Code Agents	<code>def custom_tool(args): ...</code>

https://huggingface.co/docs/smolagents/conceptual_guides/intro_agents

LLM-Agents Predecessors. WebGPT

GPT-3 Based model finetuned for answering ELI5 questions using web environment.

arXiv:2112.09332 [cs.CL]



(a) Screenshot from the demonstration interface.

```
•Question
How can I train the crows in my neighborhood to bring me gifts?

•Quotes
From Gifts From Crows | Outside My Window (www.birdsoutsidemymwindow.org)
> Many animals give gifts to members of their own species but crows and other corvids are the only ones known to give gifts to humans.

•Past actions
Search how to train crows to bring you gifts
Click Gifts From Crows | Outside My Window www.birdsoutsidemymwindow.org
Quote
Back

•Title
Search results for: how to train crows to bring you gifts

•Scrollbar: 0 - 11
•Text
[0]How to Make Friends With Crows - PetHelpfulpethelpful.com:
If you did this a few times, your crows would learn your new place, but as I said, I'm not sure if they will follow or visit you there since it's probably not in their territory. The other option is simply to make new crow friends with the crows that live in your new neighborhood.

[1]Gifts From Crows | Outside My Windowwww.birdsoutsidemymwindow.org:
The partial piece of apple may have been left behind when the crow was startled rather than as a gift. If the crows bring bright objects you'll know for sure that it's a gift because it's not something they eat. Brandi Williams says: May 28, 2020 at 7:19 am.

•Actions left: 96
•Next action
```

(b) Corresponding text given to the model.

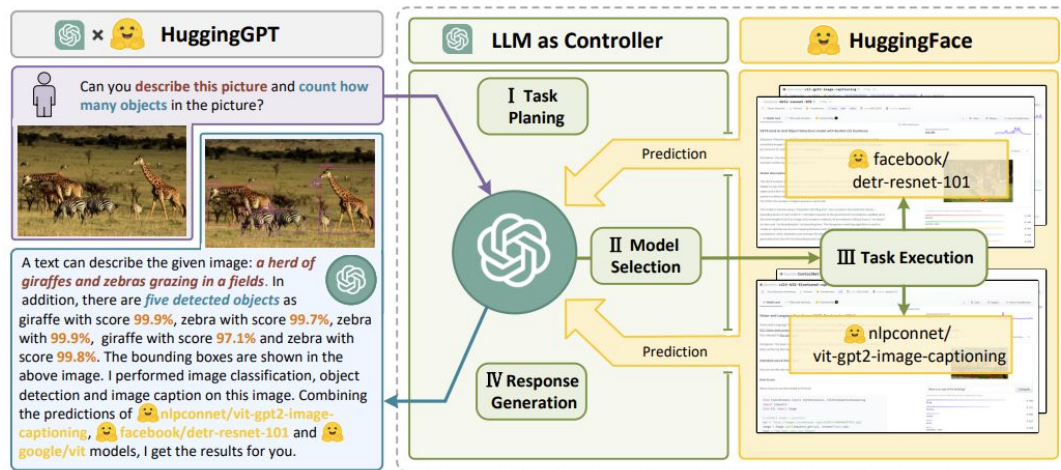
Table 1: Actions the model can take. If a model generates any other text, it is considered to be an invalid action. Invalid actions still count towards the maximum, but are otherwise ignored.

Command	Effect
Search <query>	Send <query> to the Bing API and display a search results page
Clicked on link <link ID>	Follow the link with the given ID to a new page
Find in page: <text>	Find the next occurrence of <text> and scroll to it
Quote: <text>	If <text> is found in the current page, add it as a reference
Scrolled down <1, 2, 3>	Scroll down a number of times
Scrolled up <1, 2, 3>	Scroll up a number of times
Top	Scroll to the top of the page
Back	Go to the previous page
End: Answer	End browsing and move to answering phase
End: <Nonsense, Controversial>	End browsing and skip answering phase

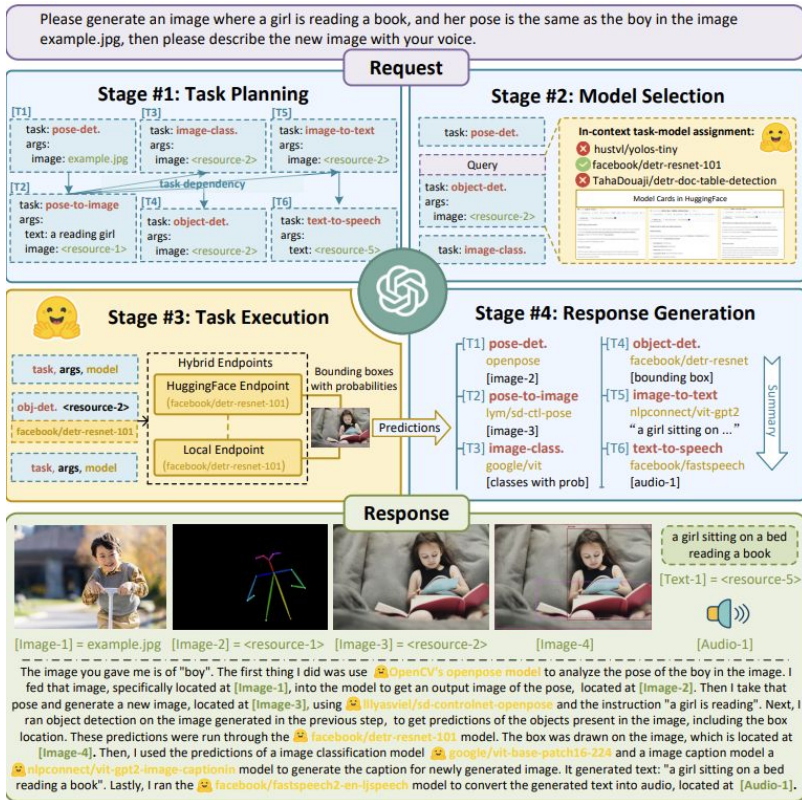
LLM-Agents Predecessors. HuggingGPT/JARVIS

HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face
arXiv:2303.17580 [cs.CL]

Shows how text-only models can interact with other models developed by the machine learning community, expanding their potential applications.



LLM-Agents Predecessors. HuggingGPT/JARVIS



Task Planning	Prompt #1 Task Planning Stage - The AI assistant performs task parsing on user input, generating a list of tasks with the following format: [{"task": task, "id": task_id, "dep": dependency, "task_ids": "args": {"text": text, "image": URL, "audio": URL, "video": URL}}]. The "dep" field denotes the id of the previous task which generates a new resource upon which the current task relies. The tag "<resource>" task_id represents the generated text, image, audio, or video from the dependency task with the corresponding task_id. The task must be selected from the following options: [{"Available Task List"}]. Please note that there exists a logical connections and order between the tasks. In case the user input cannot be parsed, an empty JSON response should be provided. Here are several cases for your reference: [{"Demonstrations"}]. To assist with task planning, the chat history is available as [{"Chat Logs"}], where you can trace the user-mentioned resources and incorporate them into the task planning stage. Demonstrations Can you tell me how many objects in e1.jpg? <pre>[{"task": "object-detection", "id": 0, "dep": [-1], "args": {"image": "e1.jpg"}}]</pre> In e2.jpg, what's the animal and what's it doing? <pre>[{"task": "image-to-text", "id": 0, "dep": [-1], "args": {"image": "e2.jpg"}}, {"task": "image-class", "id": 1, "dep": [-1], "args": {"image": "e2.jpg"}}, {"task": "object-detection", "id": 2, "dep": [-1], "args": {"image": "e2.jpg"}}, {"task": "visual-question-answering", "id": 3, "dep": [-1], "args": {"text": "what's the animal doing?", "image": "e2.jpg"}}]</pre> First generate a HED image of e3.jpg, then based on the HED image and a text "a girl reading a book", create a new image as a response. <pre>[{"task": "pose-detection", "id": 0, "dep": [-1], "args": {"image": "e3.jpg"}}, {"task": "pose-text-to-image", "id": 1, "dep": [0], "args": {"text": "a girl reading a book", "image": "<resource-0>"}}]</pre>
	Prompt #2 Model Selection Stage - Given the user request and the call command, the AI assistant helps the user to select a suitable model from a list of models to process the user request. The AI assistant merely outputs the model id of the most appropriate model. The output must be in a strict JSON format: [{"id": "id", "reason": "your detail reason for the choice"}]. We have a list of models for you to choose from [{"Candidate Models"}]. Please select one model from the list. Candidate Models <pre>[{"model_id": "model id #1", "metadata": "meta-info #1", "description": "description of model #1"}, {"model_id": "model id #2", "metadata": "meta-info #2", "description": "description of model #2"}, ... [{"model_id": "model id #K", "metadata": "meta-info #K", "description": "description of model #K"}]</pre>
Model Selection	
Response Generation	Prompt #4 Response Generation Stage - With the input and the inference results, the AI assistant needs to describe the process and results. The previous stages can be formed as - User Input: [{"User Input"}], Task Planning: [{"Tasks"}], Model Selection: [{"Model Assignment"}], Task Execution: [{"Predictions"}]. You must first answer the user's request in a straightforward manner. Then describe the task process and show your analysis and model inference results to the user in the first person. If inference results contain a file path, must tell the user the complete file path. If there is nothing in the results, please tell me you can't make it.

Table 1: The details of the prompt design in HuggingGPT. In the prompts, we set some injectable slots such as [{"Demonstrations"}] and [{"Candidate Models"}]. These slots are uniformly replaced with the corresponding text before being fed into the LLM.

Tooling

Tooling. Common Tools

Commonly Used Tools:

- **Web Search:** For accessing real-time, up-to-date information.
 - Google API
 - Serp
 - Yandex
 - Tavily: Designed specifically for LLMs
- **Code Interpreter:** For executing code (Python, etc.) to perform calculations, data analysis, or file manipulation.
 - Python sandbox
 - Bash
 - JS
 - e.t.c
- **Database Query:** To retrieve structured information from SQL or NoSQL databases.
 - Postgres
 - Clickhouse
 - e.t.c
- **External APIs:** To interact with any service (GitHub, Spotify, corporate systems).
- **Retrieval:** To search over a private knowledge base (e.g., using a vector database).
 - Confluence/Wiki
 - Vector DB over documents

Tooling. How tools are provided?

1. Python API interface:

```
from openai import OpenAI
import json

client = OpenAI()

# 1. Define a list of callable tools for the model
tools = [
    {
        "type": "function",
        "name": "get_horoscope",
        "description": "Get today's horoscope for an astrological sign.",
        "parameters": {
            "type": "object",
            "properties": {
                "sign": {
                    "type": "string",
                    "description": "An astrological sign like Taurus or Aquarius",
                },
            },
            "required": ["sign"],
        },
    },
]

def get_horoscope(sign):
    return f"{sign}: Next Tuesday you will befriend a baby otter."
```

```
# Create a running input list we will add to over time
input_list = [
    {"role": "user", "content": "What is my horoscope? I am an Aquarius."}
]
```

```
# 2. Prompt the model with tools defined
response = client.responses.create(
    model="gpt-5",
    tools=tools,
    input=input_list,
)
```

```
# Save function call outputs for subsequent requests
input_list += response.output
```

```
for item in response.output:
    if item.type == "function_call":
        if item.name == "get_horoscope":
            # 3. Execute the function logic for get_horoscope
            horoscope = get_horoscope(json.loads(item.arguments))
```

```
model
    # 4. Provide function call results to the
    input_list.append({
        "type": "function_call_output",
        "call_id": item.call_id,
        "output": json.dumps({
            "horoscope": horoscope
        })
    })
```

```
print("Final input:")
print(input_list)
```

```
response = client.responses.create(
    model="gpt-5",
    instructions="Respond only with a horoscope generated by a tool.",
    tools=tools,
    input=input_list,
)
```

```
# 5. The model should be able to give a response!
print("Final output:")
print(response.model_dump_json(indent=2))
print("\n" + response.output_text)
```

Tooling. How tools are provided?

2. OpenAI API interface:

```
{
  "model": "qwen3-32b",
  "instructions": "Respond only with a horoscope
generated by a tool.",
  "tools": [
    {
      "type": "function",
      "name": "get_horoscope",
      "description": "Get today's horoscope for an
astrological sign.",
      "parameters": {
        "type": "object",
        "properties": {
          "sign": {
            "type": "string",
            "description": "An astrological sign
like Taurus or Aquarius"
          }
        },
        "required": ["sign"]
      }
    }
  ],
}
```

```
"input": [
  {
    "role": "user",
    "content": "What is my horoscope? I am an
Aquarius."
  },
  {
    "role": "assistant",
    "content": [
      {
        "type": "function_call",
        "name": "get_horoscope",
        "arguments": "{\"sign\": \"Aquarius\"}",
        "call_id": "call_12345"
      }
    ]
  },
  {
    "type": "function_call_output",
    "call_id": "call_12345",
    "output": "{\"horoscope\": \"Aquarius: Next
Tuesday you will befriend a baby otter.\"}"
  }
]
```

Tooling. How tools are provided?

3. LLM Chat template:

```
<|im_start|>system
# Tools
```

You may call one or more functions to assist with the user query.

You are provided with function signatures within `<tools></tools>` XML tags:

```
<tools>
{
  "type": "function",
  "name": "get_horoscope",
  "description": "Get today's horoscope for an astrological sign.",
  "parameters": {
    "type": "object",
    "properties": {
      "sign": {
        "type": "string",
        "description": "An astrological sign like Taurus or Aquarius"
      }
    },
    "required": ["sign"]
  }
}
</tools>
```

For each function call, return a json object with function name and arguments within

`<tool_call></tool_call>` XML tags:

```
<tool_call>
{"name": "<function-name>", "arguments":
<args-json-object>}
</tool_call>
<|im_end|>
<|im_start|>user
What is my horoscope? I am an Aquarius.
```

```
<|im_end|>
<|im_start|>assistant
<tool_call>
{"name": "get_horoscope", "arguments": {"sign":
"Aquarius"}}
</tool_call>
```

```
<|im_end|>
<|im_start|>user
<tool_response>
{"call_id": "call_12345", "output": {"horoscope":
"Aquarius: Next Tuesday you will befriend a baby otter."}}
</tool_response>
```

```
<|im_end|>
<|im_start|>system
Respond only with a horoscope generated by a tool.
<|im_end|>
<|im_start|>assistant
Aquarius: Next Tuesday you will befriend a baby otter.
```

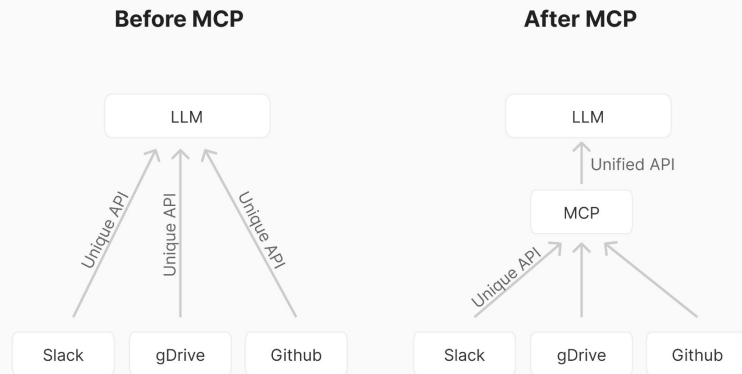
Tooling. MCP

MCP is a universal connector for model to access tools (and other stuff) developed by Anthropic.

Gained its popularity by convenient connecting models and tools.

Key advantages:

1. Huge amount of community servers
2. Standardization
3. Easy to use
4. All major agentic frameworks support MCP
5. Time to market reduction

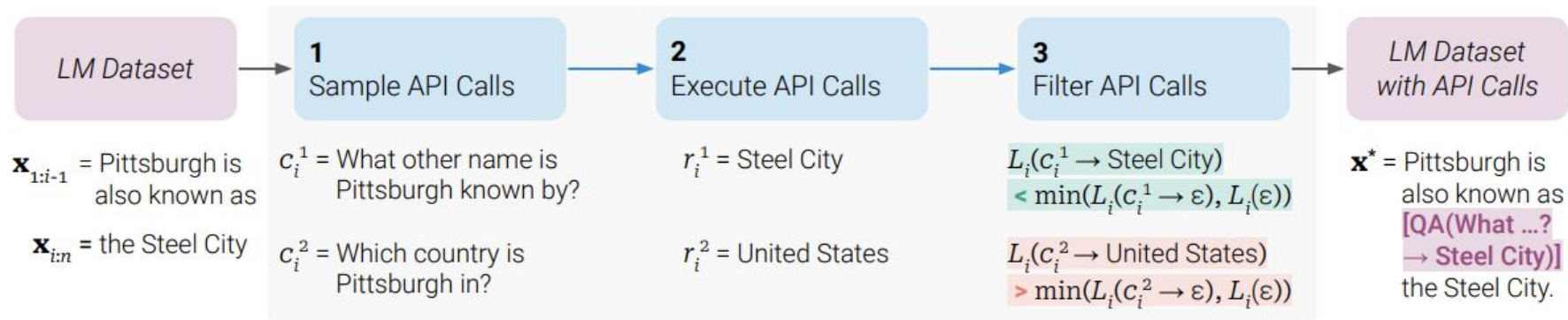


How do we teach a model to use tools?

Tooling. Training models to use tools. Toolformer

Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761 [cs.CL]

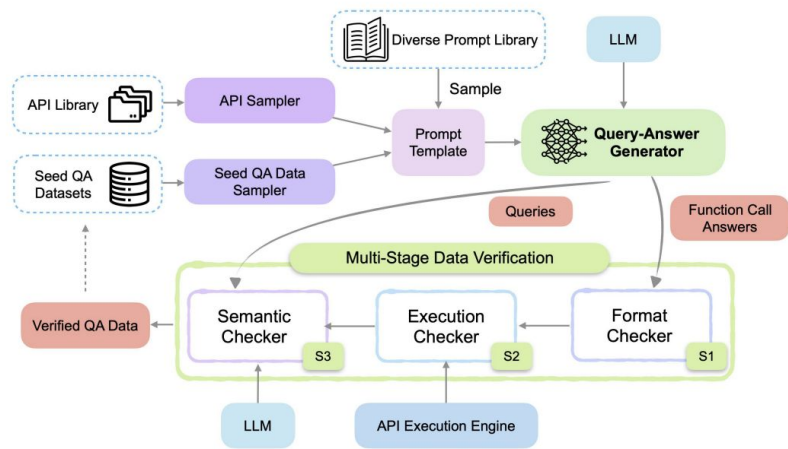
TLDR: GPT-J (6.7B) fine-tuned on a synthetic dataset with api calls created from plain text data using LLM markup



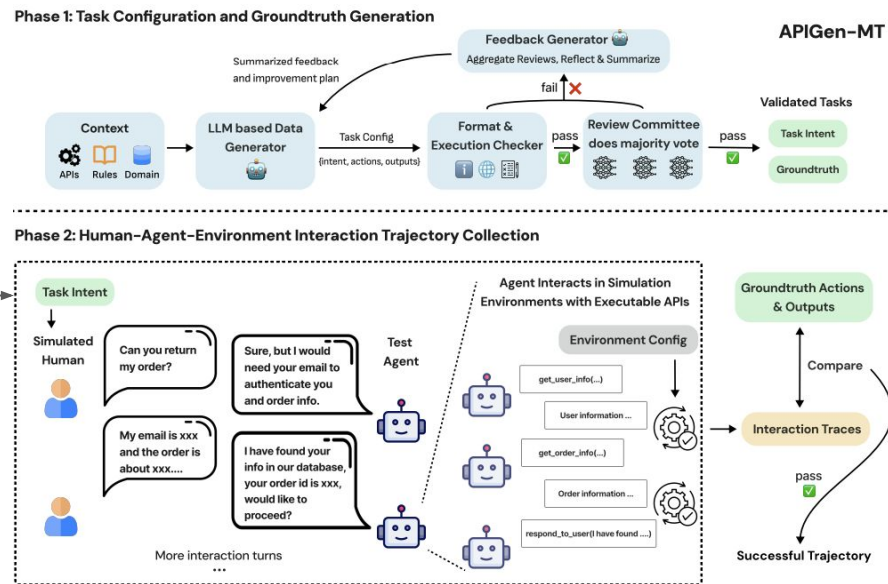
Tooling. Training models to use tools. APIGen-MT & xLam

APIGen-MT: Agentic Pipeline for Multi-Turn Data Generation via Simulated Agent-Human Interplay
arXiv:2504.03601 [cs.CL]

APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets.
arXiv:2406.18518 [cs.CL]



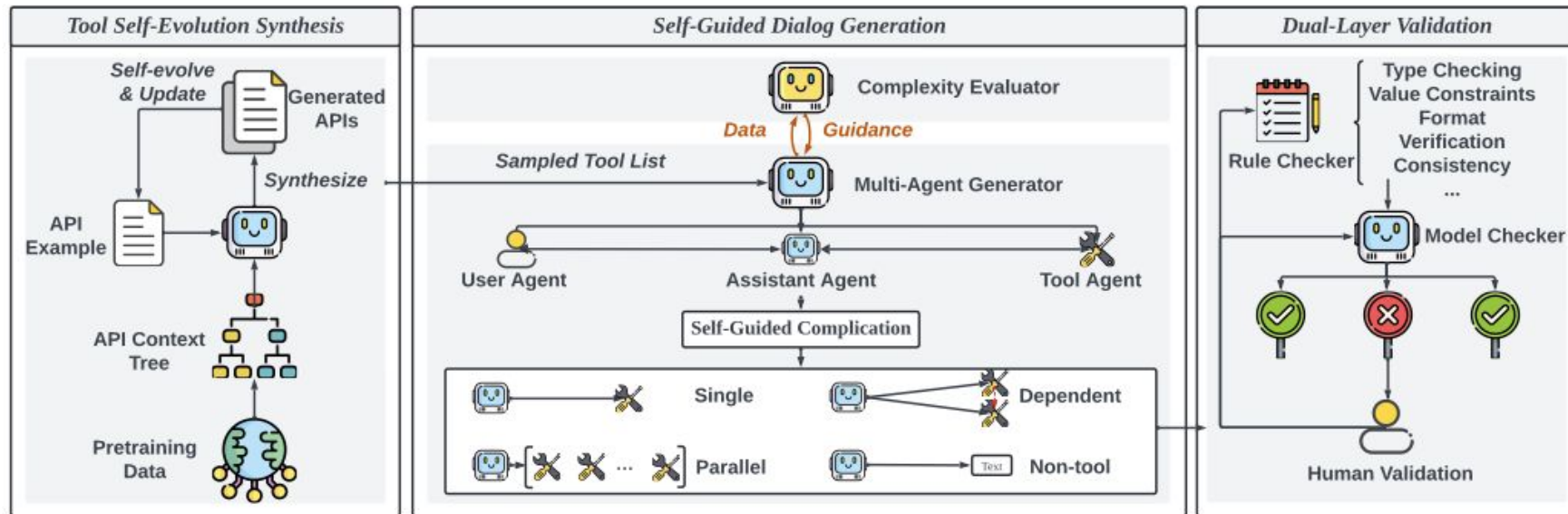
APIGen



APIGen-MT

Tooling. Training models to use tools. ToolACE

ToolACE: Winning the Points of LLM Function Calling
arXiv:2409.00920 [cs.LG]

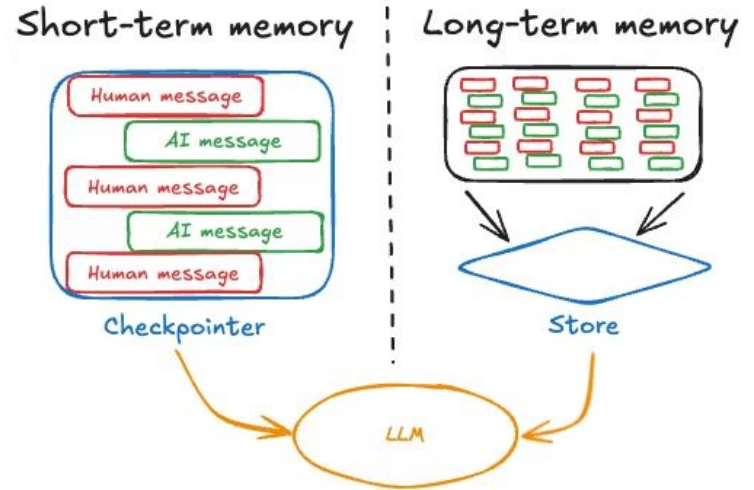


Memory. Definition, motivation and types

Memory is a system that remembers information about previous interactions.

Motivation:

For AI agents, memory is crucial because it lets them remember previous interactions, learn from feedback, and adapt to user preferences.



Memory. More on long-term memory

Subtypes:

- Episodic Memory: This type remembers specific events from the past like a user's date of birth that was used during an earlier conversation. The agent can use this memory as context in future interactions.
- Semantic Memory: This holds general knowledge about the world or things the AI has learned through past interactions. The agent can refer to this information to handle new problems effectively.
- Procedural Memory: Here the agent stores “how-to” steps or rules for making decisions. For example, it might remember the process for solving a math problem and use the same steps when tackling a similar task later.

Agent vs Multi-Step Agent

Vanilla agent might have a number of drawbacks:

- Calls tools one by one
- Might not have reasoning capabilities
- Limited reflection opportunities
- Overwhelming instructions and memory

How to fix them?

Multi-step Agents come in place

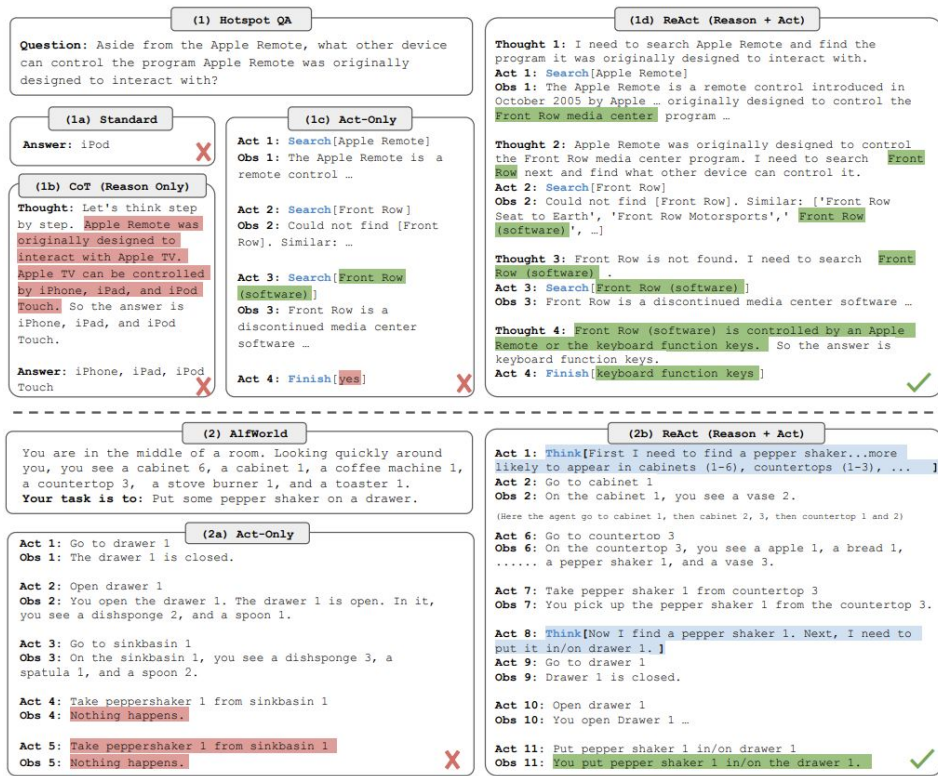
Multi-step agents. ReAct

ReAct: Synergizing Reasoning and Acting in Language Models:

Mostly prompting and response processing technique.

Main idea:

Let LLM reason first, perform some actions and then observe the outcome



ReAct example. Turn 1

```
{
  "messages": [
    {
      "role": "system",
      "content": 'Answer the following questions as best you can. You have access to the following tools:\n\nSearch: A search engine. Useful for when you need to answer questions about current events. Input should be a search query.\n\nCalculator: Useful for when you need to answer questions about math.\n\nThe way you use the tools is by specifying a json blob.\n\nSpecifically, this json should have a action key (with the name of the tool to use) and a action_input key (with the input to the tool going here).\n\nThe only values that should be in the "action" field are: Search, Calculator\n\nThe $JSON_BLOB should only contain a SINGLE action, do NOT return a list of multiple actions. Here is an example of a valid $JSON_BLOB:\n\n{\n  "action": $TOOL_NAME,\n  "action_input": $INPUT\n}\n\nALWAYS use the following format:\n\nQuestion: the input question you must answer\nThought: you should always think about what to do\nAction: \n\n$JSON_BLOB\n\nObservation: the result of the action\n... (this Thought/Action/Observation can repeat N times)\nThought: I now know the final answer\nFinal Answer: the final answer to the original input question\n\nBegin! Reminder to always use the exact characters Final Answer when responding.',
    },
    {
      "role": "user",
      "content": "Who is Olivia Wilde's boyfriend? What is his current age raised to the 0.23 power?\n\n",
    },
  ],
  "model": "gpt-3.5-turbo",
  "request_timeout": 60,
  "max_tokens": None,
  "stream": False,
  "n": 1,
  "temperature": 0.0,
  "stop": ["Observation:"],
}
```

ReAct example. Turn 2

Who is Olivia Wilde's boyfriend? What is his current age raised to the 0.23 power?

This was your previous work (but I haven't seen any of it! I only see what you return as final answer):

Thought: I need to use a search engine to find Olivia Wilde's boyfriend and a calculator to raise his age to the 0.23 power.

Action:

```
{  
  "action": "Search",  
  "action_input": "Olivia Wilde boyfriend"  
}
```

Observation: Sudeikis and Wilde's relationship ended in November 2020. Wilde was publicly served with court documents regarding child custody while she was presenting Don't Worry Darling at CinemaCon 2022. In January 2021, Wilde began dating singer Harry Styles after meeting during the filming of Don't Worry Darling.

Thought:

ReAct example. Turn 3

Who is Olivia Wilde's boyfriend? What is his current age raised to the 0.23 power?

This was your previous work (but I haven't seen any of it! I only see what you return as final answer):

Thought: I need to use a search engine to find Olivia Wilde's boyfriend and a calculator to raise his age to the 0.23 power.

Action:

```
{  
  "action": "Search",  
  "action_input": "Olivia Wilde boyfriend"  
}
```

Observation: Sudeikis and Wilde's relationship ended in November 2020. Wilde was publicly served with court documents regarding child custody while she was presenting Don't Worry Darling at CinemaCon 2022. In January 2021, Wilde began dating singer Harry Styles after meeting during the filming of Don't Worry Darling.

Thought: I need to use a search engine to find Harry Styles' current age.

Action:

```
{  
  "action": "Search",  
  "action_input": "Harry Styles age"  
}
```

Observation: 29 years

Thought:

ReAct example. Final turn

Who is Olivia Wilde's boyfriend? What is his current age raised to the 0.23 power?

This was your previous work (but I haven't seen any of it! I only see what you return as final answer):

Thought: I need to use a search engine to find Olivia Wilde's boyfriend and a calculator to raise his age to the 0.23 power.

Action:

```
{  
  "action": "Search",  
  "action_input": "Olivia Wilde boyfriend"  
}
```

Observation: Sudeikis and Wilde's relationship ended in November 2020. Wilde was publicly served with court documents regarding child custody while she was presenting Don't Worry Darling at CinemaCon 2022. In January 2021, Wilde began dating singer Harry Styles after meeting during the filming of Don't Worry Darling.

Thought: I need to use a search engine to find Harry Styles' current age.

Action:

```
{  
  "action": "Search",  
  "action_input": "Harry Styles age"  
}
```

Observation: 29 years

Thought: Now I need to calculate 29 raised to the 0.23 power.

Action:

```
{  
  "action": "Calculator",  
  "action_input": "29^0.23"  
}
```

Observation: Answer: 2.169459462491557

Thought:

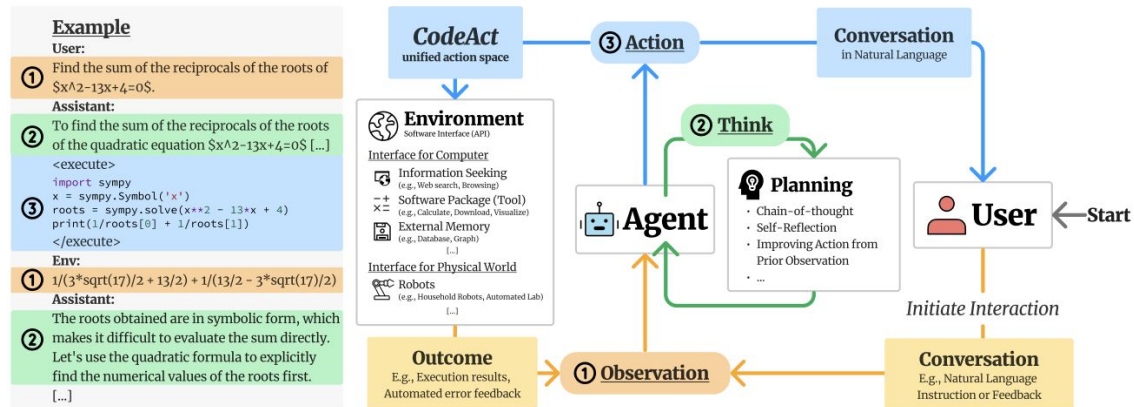
LLM Response:

I now know the final answer.

Final Answer: 2.169459462491557

Multi-step agents. CodeAct

What if we replace json-like tool calls with code executions with python code but still leave a room for reasoning and natural language conversations?





Here is a dataset about auto MPG: "https://huggingface.co/datasets/scikit-learn/auto-mpg/raw/main/auto-mpg.csv". You should download and load the dataset from the URL, take a look at it, check for missing values, do a train-test split, and train a regression model predicting its MPG. You should use everything except "mpg" and "car name" as the input features.

CodeActAgent

To accomplish this task, I will use the pandas library to download and load the dataset, check for missing values, and perform a train-test split. Then, I will use the scikit-learn library to train a regression model.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Download and load the dataset
df = pd.read_csv(
    "https://huggingface.co/datasets/scikit-learn/auto-mpg/raw/main/auto-mpg.csv")
# Check for missing values
print(df.isnull().sum())
# Remove rows with missing values
df = df.dropna()
# Perform train-test split
X = df.drop(columns=["mpg", "car name"])
y = df["mpg"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)

# Train a regression model
model = LinearRegression()
model.fit(X_train, y_train)
# Predict MPG for test data
y_pred = model.predict(X_test)
# Evaluate the model
print("Mean squared error: ", np.mean((y_test - y_pred) ** 2))
print("R^2 score: ", r2_score(y_test, y_pred))
```

Use Pandas Library to Download, Examine, and Process Data

Use Scikit-Learn Library to Split Data and Train Regression Model

Environment

```
mpg      0
...omitted for space...
car name 0
dtype: object

ValueError                                Traceback (most recent call last)
~/tmp/ipykernel_12/1372561217.py in ?()
    18 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2 [...])
    19
    20 # Train a regression model
    21 model = LinearRegression()
--> 22 model.fit(X_train, y_train)
    23
    24 # Predict MPG for test data
    25 y_pred = model.predict(X_test)

... [omitted for space] ...

ValueError: could not convert string to float: '?'
```

Self-Debug from Automated Feedback

CodeActAgent

It seems that there is a data type issue in the dataset. The '?' character is present in the data, which is causing the error when trying to convert the data to a float. To fix this, I will remove the rows containing the '?' character.

```
# Remove rows containing '?'
df = df.replace('?', np.nan)
df = df.dropna()
X = df.drop(columns=["mpg", "car name"])
y = df["mpg"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Mean squared error: ", np.mean((y_test - y_pred) ** 2))
print("R^2 score: ", r2_score(y_test, y_pred))
```

Environment

Mean squared error: 10.718864418838367
R^2 score: 0.790150838676835

CodeActAgent

The mean squared error is 10.711 and the R² score is 0.790, which indicates that the model has a good fit with the data [...]

User

Can you also give me the MSE and R² for the training set?

CodeActAgent

Able to Answer follow-up Question Interactively

Certainly! Here are the mean squared error (MSE) and R² scores [...]

```
from sklearn.metrics import mean_squared_error
# Calculate MSE for training set
mse_train = mean_squared_error(y_train, model.predict(X_train))
# Calculate R^2 score for training set
r2_train = r2_score(y_train, model.predict(X_train))

print("Mean squared error (training set):", mse_train)
print("R^2 score (training set):", r2_train)
```

Environment

Mean squared error (training set): 10.979255842373478
R² score (training set): 0.826001578671867

CodeActAgent

The mean squared error for the training set is 10.979 and the R² score is 0.826, which indicates [...]

User

Can you visualize the regression coefficient? You may access them via model.coef_ and their names via model.feature_names_in_

CodeActAgent

Certainly! Here is a visualization of the regression coefficients [...]

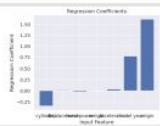
```
import matplotlib.pyplot as plt
# Get regression coefficients
coef = model.coef_

# Get names of input features
feature_names = model.feature_names_in_
# Plot regression coefficients
plt.bar(feature_names, coef)
plt.xlabel('Input Feature')
plt.ylabel('Regression Coefficient')
plt.title('Regression Coefficients')
# Display the plot
plt.show()
```

Use Matplotlib Library for Data Visualization

Environment

<Figure size 640x480 with 1 Axes>



CodeActAgent

The regression coefficients are plotted as a bar chart, with each input feature on the x-axis and the corresponding coefficient on the y-axis [...]



Can you rotate the x ticks by 45 degrees for better readability?

CodeActAgent

Certainly! Here is the updated bar chart with the x-ticks rotated by 45 degrees for better readability:

```
[...same code omitted...]
# Plot regression coefficients
plt.bar(feature_names, coef, tick_labels=False)
plt.xticks(feature_names, feature_names,
           rotation=45, ha='right', fontsize=12)
[...same code omitted...]
```

Environment

```
<Figure size 640x480 with 1 Axes>

AttributeError                                Traceback (most recent call last)
Cell In[3], line 18
      7 feature_names = model.feature_names_in_
      8 # Plot regression coefficients
--> 10 plt.bar(feature_names, coef, tick_labels=False)
     11 plt.xticks(feature_names, feature_names,
     12           rotation=45, ha='right', fontsize=12)
     13 plt.xlabel('Input Feature')
     14
     15 [...]
```

Self-Debug for Data Visualization

CodeActAgent

It seems that the tick_labels argument is not a valid argument for the bar() function in Matplotlib. Instead, we can use the tick_params() function to rotate the x-ticks [...]

```
[...same code omitted...]
plt.bar(feature_names, coef)
plt.tick_params(axis='x',
               labelrotation=45, ha='right', fontsize=12)
[...same code omitted...]
```

[3 rounds of self-debugging omitted for space]

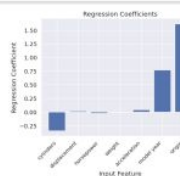
CodeActAgent

I apologize for the confusion. It seems that the set_xticks() function is not a valid function in Matplotlib. Instead, we can use the xticks() function to set the x-ticks and rotate them by 45 degrees for better readability. Here is the updated bar chart with the x-ticks rotated by 45 degrees:

```
[...same code omitted...]
plt.xticks(feature_names, feature_names,
           rotation=45, ha='right', fontsize=12)
[...same code omitted...]
```

Environment

<Figure size 640x480 with 1 Axes>



User

[Stops Interaction]

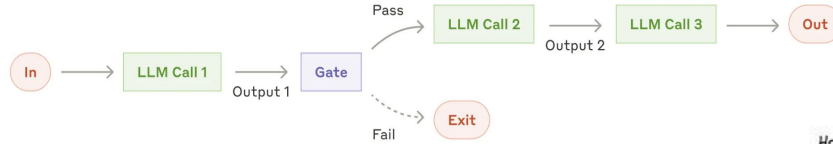
Multi-Agent Systems. Why do we need them?

In single agent scenarios one might face several issues:

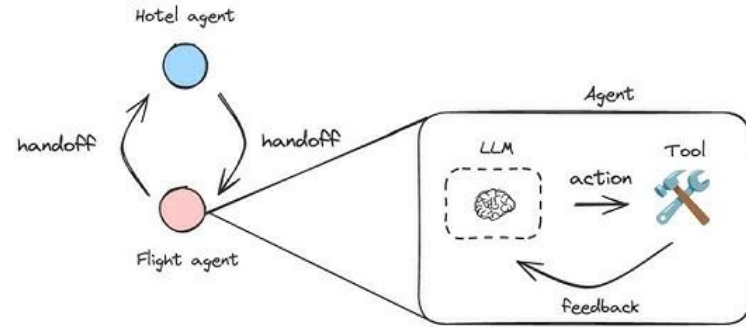
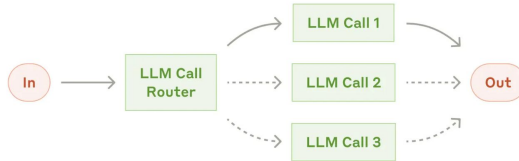
1. Cannot change agents system prompt/model.
2. Single powerful agent is harder to fine-tune than a small piece of MAS.
3. Model overwhelming with tools and responsibility in general.
4. Usually single general-purpose agents require bigger models.

Multi-Agent Systems. Control and context flow

Workflow Agent



LLM Agent



Main building blocks:

Workflow Agent: Fixed order of agent invocations

LLM Agent: Model decides whether to use another agent or not

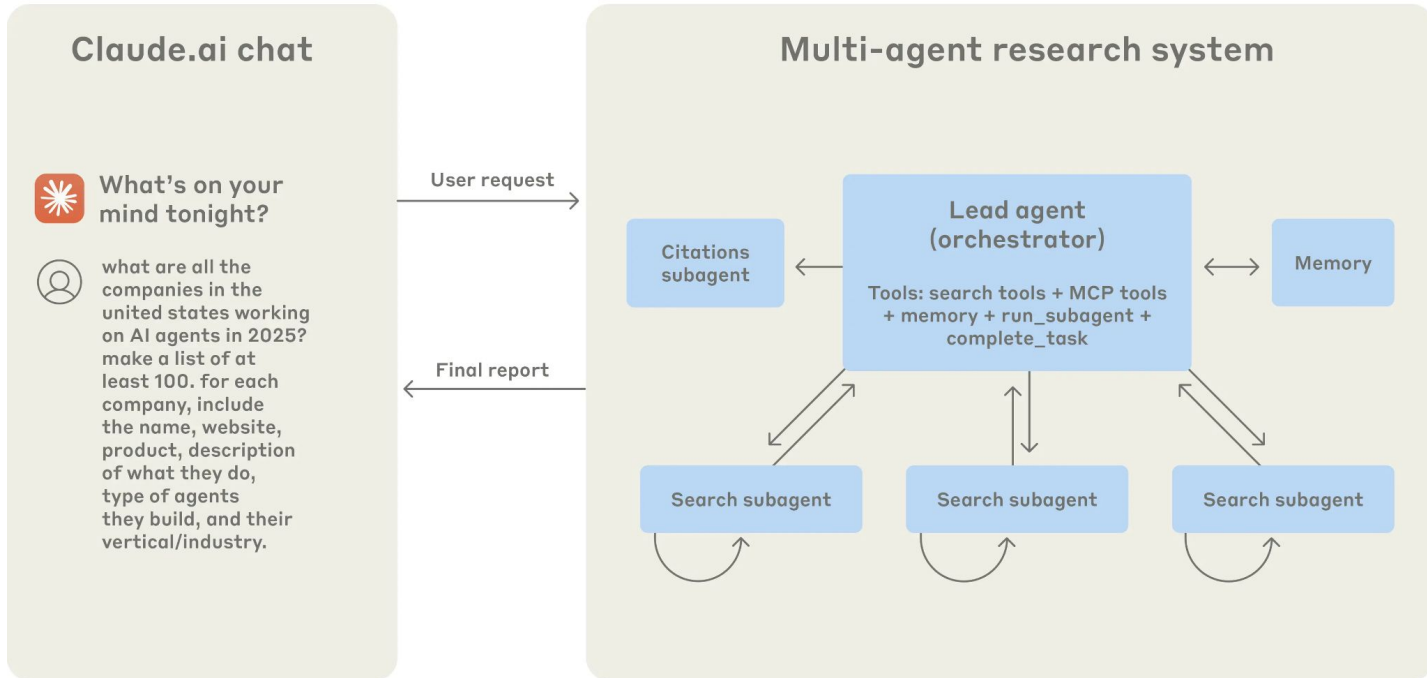
Control flow types:

Handoff - Full control and context transfer to another agent

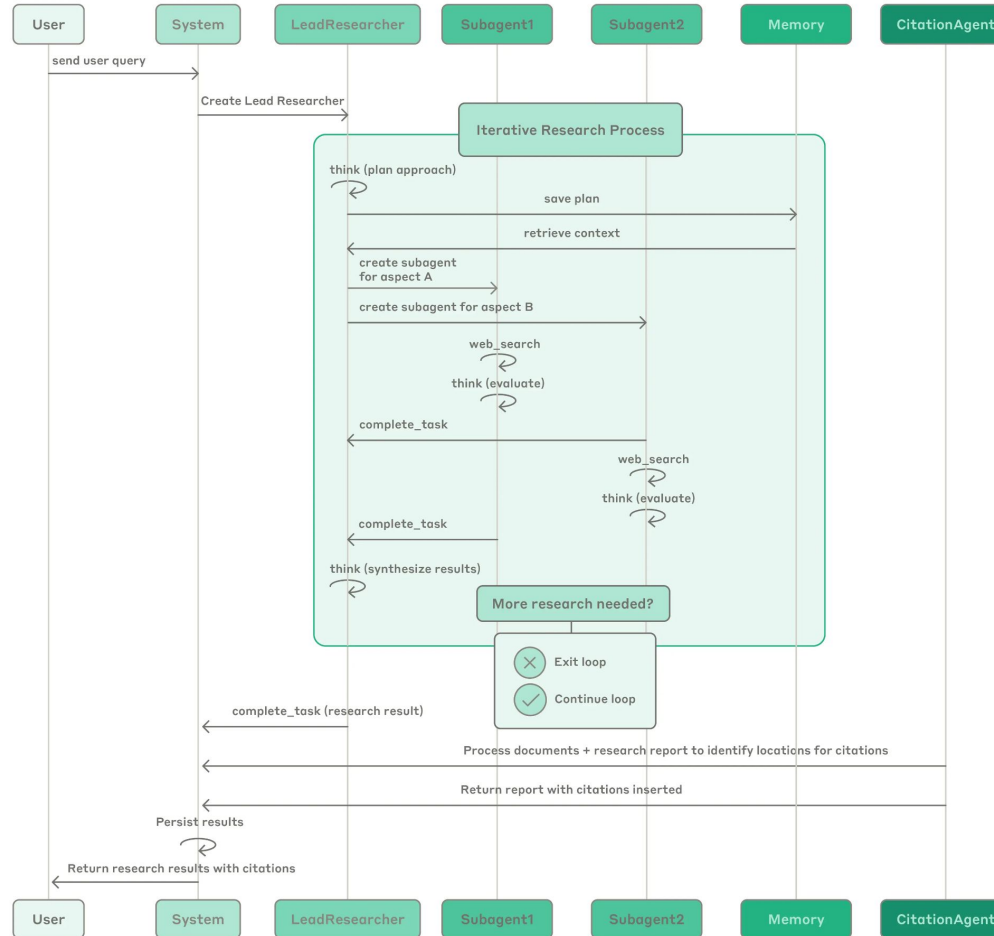
Agent-as-tool - Call the agent and return his response back immediately. Usually no context is passed

MAS Example

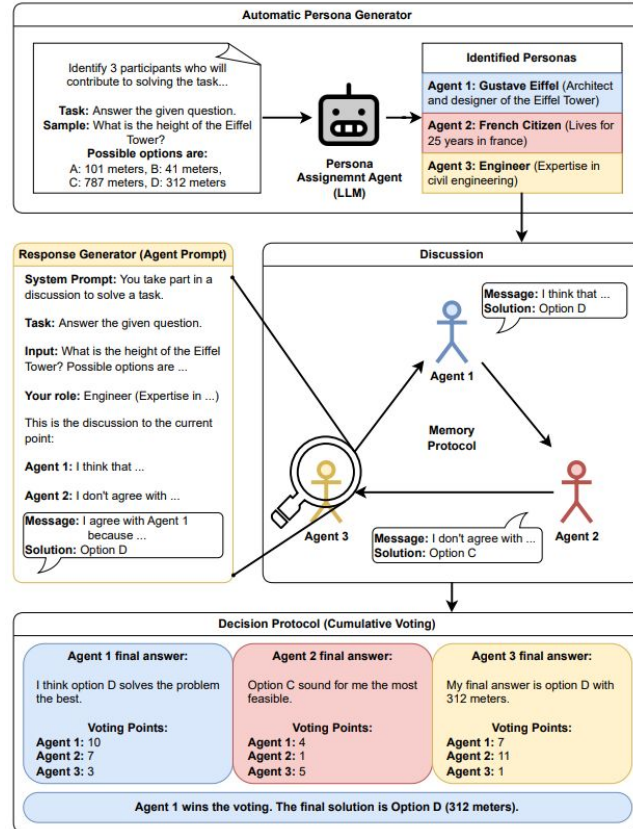
High-level Architecture of Advanced Research



Multi-agent System Process Diagram



Agents Collaboration. Voting and debate

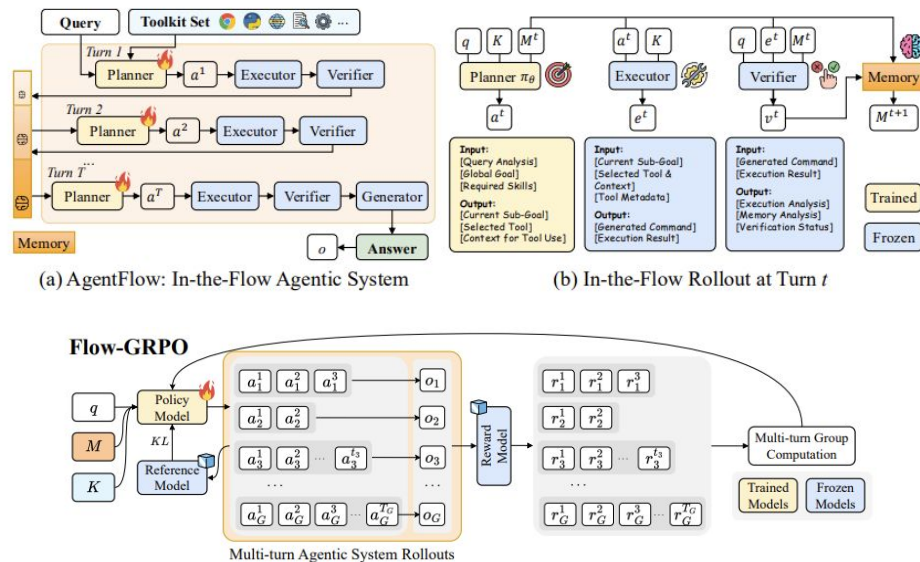


Voting or Consensus?
Decision-Making in Multi-Agent
Debate. Kaesberg et. al.

Training agents. AgentFlow & FlowGRPO

The paper introduces a new way to train modular tool-using LLM agents for complex multi-step reasoning tasks:

- AgentFlow: Planner, Executor, Verifier, and Generator communicating via a memory.
- “in the flow” on-policy training with only a final binary reward per trajectory.
- Flow-GRPO: GRPO-based RL algorithm that assigns the same terminal reward to all steps in a trajectory with group-normalized advantages.
- LLM judge to score final answers
- 7B AgentFlow model trained with Flow-GRPO outperforms GPT-4o on multiple tool-use and reasoning benchmarks



Training agents. AgentFlow & FlowGRPO

Algorithm 1 In-the-Flow Optimization for AGENTFLOW

Require: Dataset $\mathcal{D}$, Action Planner policy π_θ , Tool Executor $\mathcal{E}$, Executive Verifier $\mathcal{V}$, Solution Generator $\mathcal{G}$, Toolset K , and Shared Evolving Memory M

Ensure: Optimized Action Planner parameters θ^*

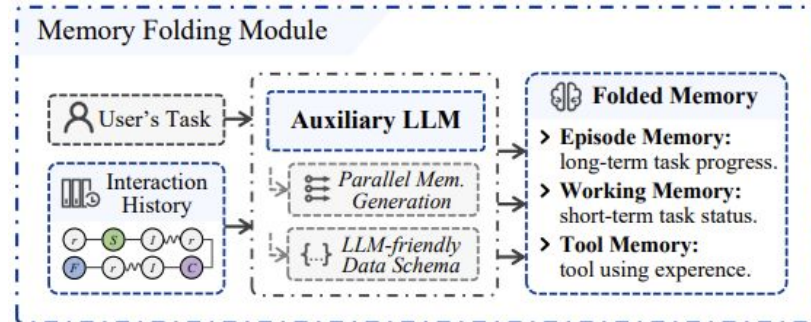
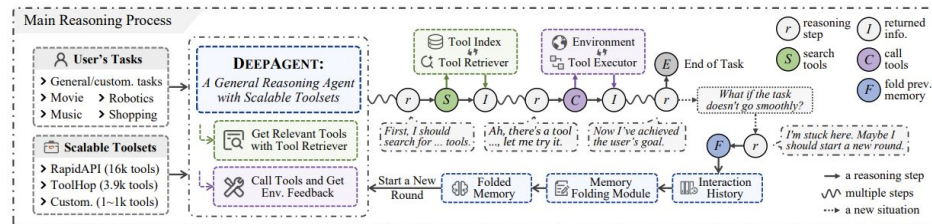
```
1: for each training iteration do
2:   for each query-label pair  $(q, y^*) \sim \mathcal{D}$  do
3:     1. IN-THE-FLOW ROLLOUT GENERATION
4:     Initialize:  $t \leftarrow 1, M^t \leftarrow q$ 
5:     repeat
6:        $a^t \sim \pi_\theta(a^t \mid q, K, M^t)$  {Plan Action}
7:        $e^t \sim \mathcal{E}(e^t \mid a^t, K)$  {Execute Action}
8:        $v^t \sim \mathcal{V}(v^t \mid q, e^t, M^t)$  {Verify Result}
9:        $M^{t+1} = f_{\text{mem}}(M^t, a^t, e^t, v^t)$  {Update Memory}
10:       $t \leftarrow t + 1$ 
11:   until termination condition met
12:    $o \sim \mathcal{G}(o \mid q, M^T)$  {Generate Final Solution}
13:   2. REWARD COMPUTATION
14:    $R(a^t) = \bar{R}(o, q, y^*), \quad \forall t = 1, \dots, T$ 
15:   3. POLICY UPDATE
16:   Update the Action Planner policy  $\pi_\theta$  by maximizing the Flow-GRPO objective (Eq. 5)
17:   end for
18: end for
19: return optimized parameters  $\theta^*$ 
```

Training agents. DeepAgent. General architecture

DeepAgent introduces a unified agent that controls its own reasoning, tools, and memory for long, complex tasks.

Architecture & mechanisms

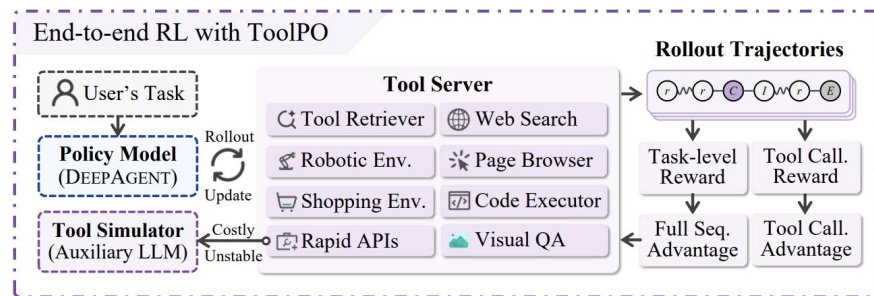
- Single reasoning stream: thoughts and tool search, tool calls, and memory folding (no external ReAct-style loops).
- Toolset retrieval: a retriever maps text queries to relevant APIs on demand.
- Structured memory folding: the agent periodically condenses history into compact JSON
- Unified tool interface: all tools plug into a common `<tool_search>` / `<tool_call>` / `<*_result>` protocol, making the system easily extensible.
- Main reasoning and non-thinking auxiliary LLM



Training agents. DeepAgent. ToolPO

Training

- Tool simulator environment: a separate LLM mimics real APIs, enabling cheap and stable RL over diverse tool-use scenarios.
- ToolPO algorithm: a PPO-style objective tailored specifically to tool-use and memory operations.
- Split rewards: task-level rewards for final success plus action-level rewards for the quality of individual tool and memory operations.
- Dual advantage attribution: global advantage is applied to all tokens, while action advantage is added only on tool/memory tokens.
- RL fine-tuning: the model is optimized on these signals, yielding more efficient and reliable tool use than standard agent baselines.



Agentic RAG. Single Agent RAG

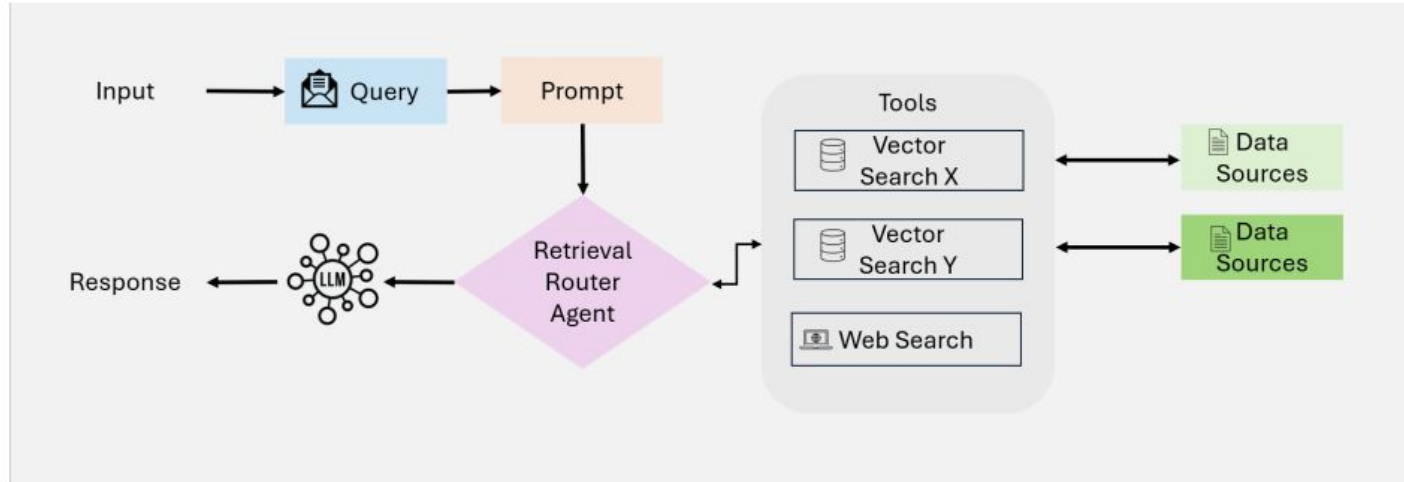


Figure 16: An Overview of Single Agentic RAG

Agentic RAG. Multi-Agent RAG

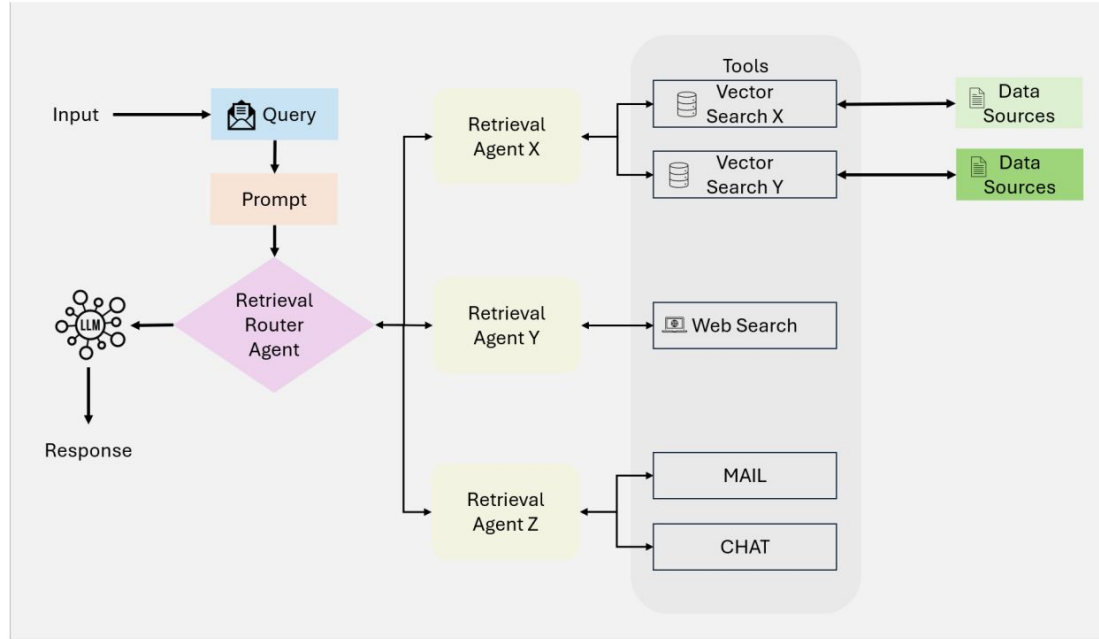


Figure 17: An Overview of Multi-Agent Agentic RAG Systems

Adaptive Agentic RAG.

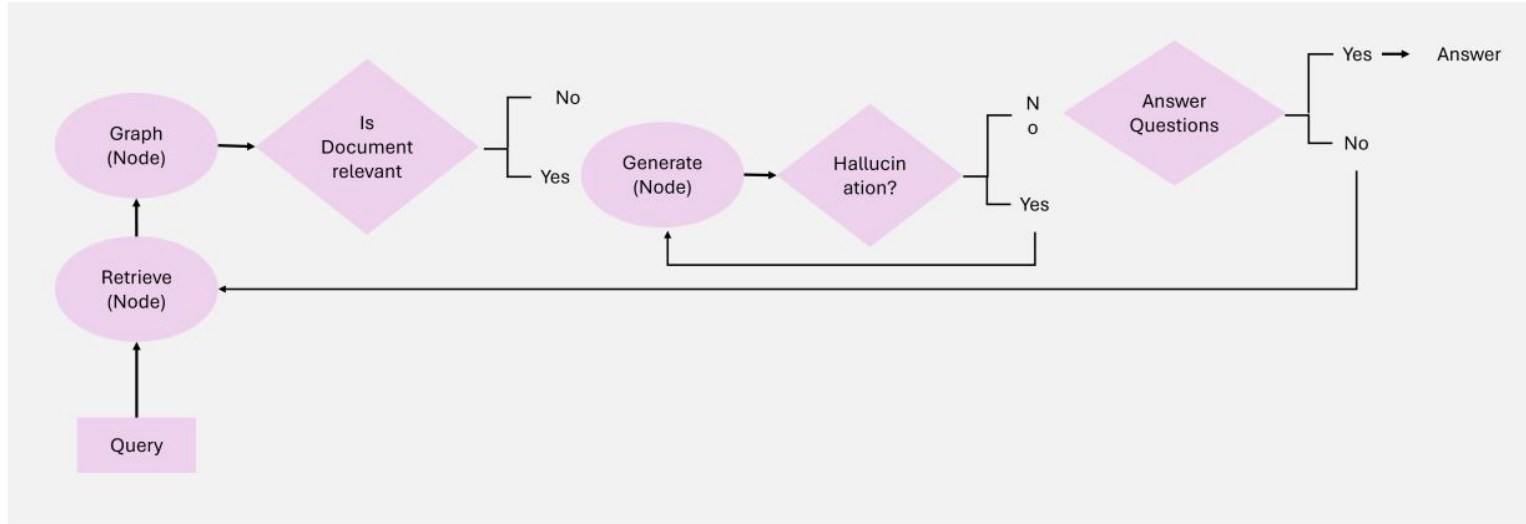


Figure 20: An Overview of Adaptive Agentic RAG

Eval. Function calling

- BFCL (Berkeley Function Calling Leaderboard) - the most popular benchmark of function calling
- ToolBench: Massive tool-use benchmark with thousands of real REST APIs
- API-Bank: Smaller but clean benchmark, tens of APIs
- RestBench–TMDB/Spotify: Narrow domain focused benchmarks (video/audio related tools)
- ToolHop: Purpose-built for multi-hop tool-use: each query requires several interdependent tool calls
- GAIA (General AI Assistant benchmark): Realistic assistant tasks (code, web search, data files, multimodal)

Eval. Agents

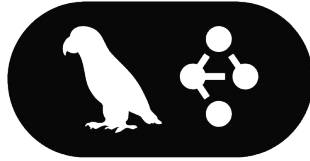
- GAIA (General AI Assistant benchmark): Realistic assistant tasks (code, web search, data files, multimodal)
- ALFWorld: Text-based household environment where an agent completes tasks like “wash the apple and put it in the fridge” via stepwise commands.
- WebShop: Simulated e-commerce site and natural-language shopping instructions like “buy cheap black running shoes with rating > 4.5.”
- τ -bench (Tau-bench): Domain-specific tasks (e.g., airline/retail) with realistic APIs and policies where the agent must converse, ask clarification questions, and call tools.
- τ^2 -bench (Tau²-bench): Extension of τ -bench to dual-control settings where both agent and user have tools and jointly affect the environment state.

Popular Agentic Frameworks



crewai

Agno

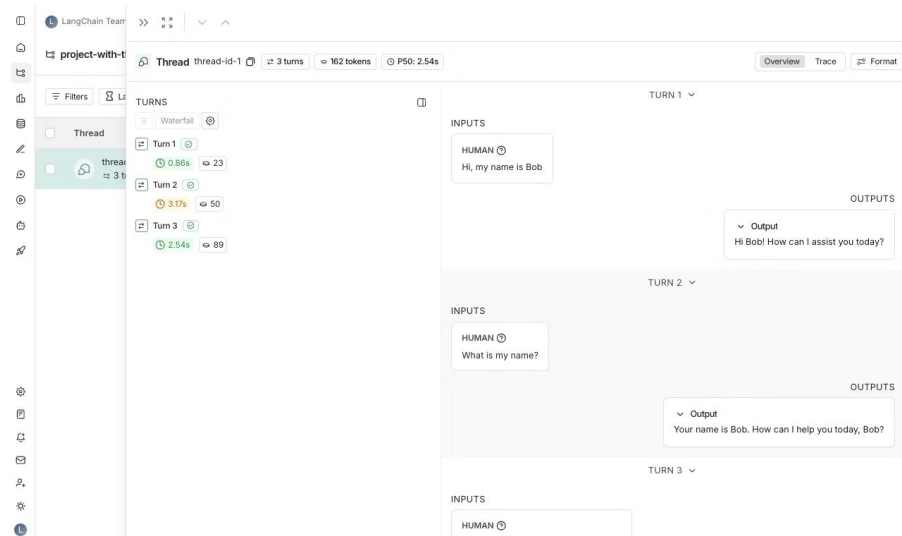
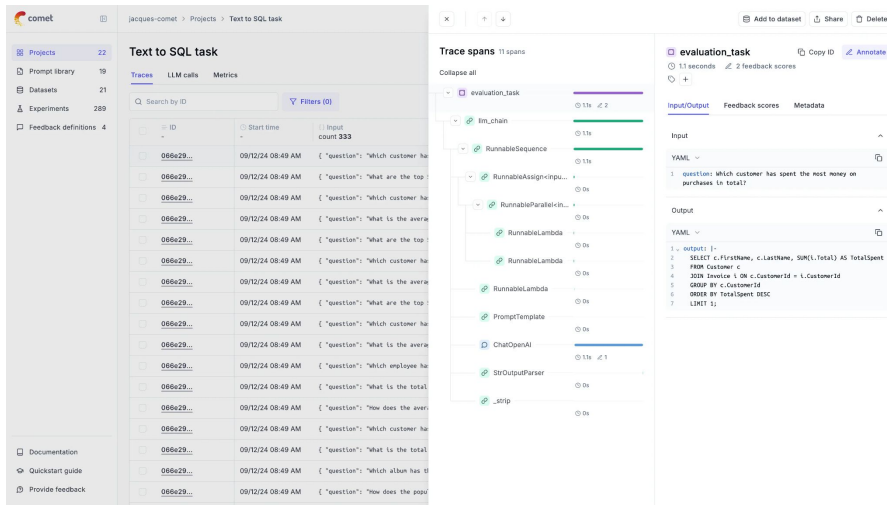


LangGraph



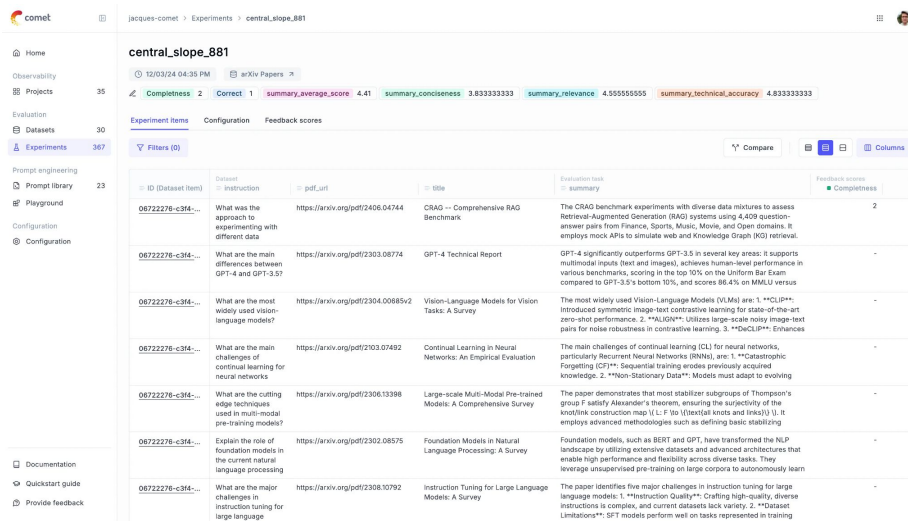
smolagents

CometML Opik



LangSmith

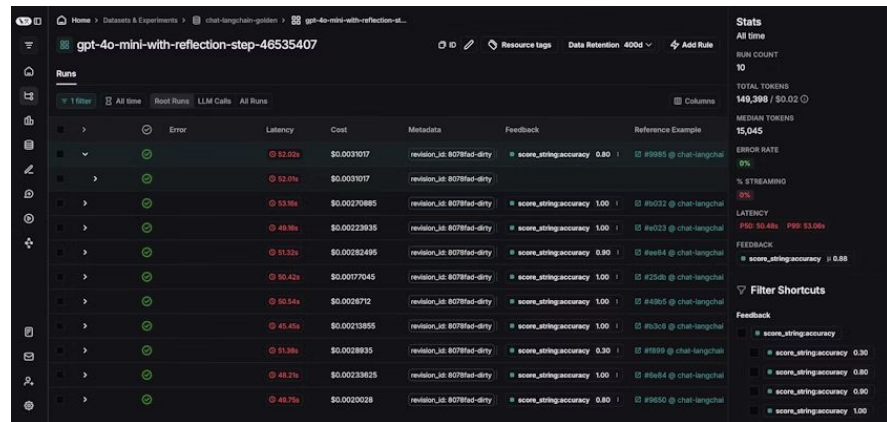
AgentOps platforms. Evaluation



The image shows the CometML Opik interface for an experiment named 'central_slope_881'. The top navigation bar includes 'Home', 'Observability', 'Projects', 'Evaluation', 'Datasets', and 'Experiments'. The 'Experiments' section is active, showing a list of experiments with columns for 'Completeness', 'Correct', 'Summary Average Score', 'Summary Conciseness', 'Summary Relevance', and 'Summary Technical Accuracy'. The 'Experiments' table shows a single experiment with a score of 4.41. Below the table, there is a 'Filter' button and a 'Compare' button. The main content area displays a table of experiment results with columns for 'ID (Dataset Item)', 'Instruction', 'pdf_url', 'Title', 'Evaluation task', and 'Feedback score'. The table lists several experiments, including '0872278-c384--', '0872278-c384--', '0872278-c384--', '0872278-c384--', '0872278-c384--', '0872278-c384--', '0872278-c384--', '0872278-c384--', and '0872278-c384--'. The table also includes a 'Feedback score' column with a 'Completeness' sub-column.

ID (Dataset Item)	Instruction	pdf_url	Title	Evaluation task	Feedback score
0872278-c384--	What was the approach to experimenting with different data	https://arxiv.org/pdf/2406.04744	CRAG -- Comprehensive RAG Benchmark	The CRAG benchmark experiments with diverse data mixtures to assess Retrieval-Augmented Generation (RAG) systems using 4,409 question-answer pairs from Finance, Sports, Music, Movie, and Open domains. It employs mock APIs to simulate web and Knowledge Graph (KG) retrieval.	2
0872278-c384--	What are the main differences between GPT-4 and GPT-3.5?	https://arxiv.org/pdf/2303.08774	GPT-4 Technical Report	GPT-4 significantly outperforms GPT-3.5 in several key areas: It supports multimodal inputs (text and images), achieves human-level performance in various benchmarks, scoring in the top 10% on the Uniform Bar Exam compared to GPT-3.5's bottom 10%, and scores 86.4% on MMLU versus	-
0872278-c384--	What are the most widely used vision-language models?	https://arxiv.org/pdf/2304.06855v2	Vision-Language Models for Vision Tasks: A Survey	The most widely used Vision-Language Models (VLMs) are: 1. **CLIP** - Introduced symmetric image-text contrastive learning for state-of-the-art zero-shot performance. 2. **ALIGN** - Utilizes large-scale noisy image-text pairs for noise robustness in contrastive learning. 3. **DeCLIP** - Enhances	-
0872278-c384--	What are the main challenges of continual learning for neural networks	https://arxiv.org/pdf/2703.07492	Continual Learning in Neural Networks: An Empirical Evaluation	The main challenges of continual learning (CL) for neural networks, particularly Recurrent Neural Networks (RNNs), are: 1. **Catastrophic Forgetting (CF)**: Sequential training erodes previously acquired knowledge. 2. **Non-Stationary Data** - Models must adapt to evolving	-
0872278-c384--	What are the cutting edge techniques used in multi-modal pre-training models?	https://arxiv.org/pdf/2306.13398	Large-scale Multi-Modal Pre-trained Models: A Comprehensive Survey	The paper demonstrates that most stabilizer subgroups of Thompson's group F satisfy Alexander's theorem, ensuring the surjectivity of the knotline construction map $L(F)$ to l -tential knots and links l . It employs advanced methodologies such as defining basic stabilizing	-
0872278-c384--	Explain the role of foundation models in the current natural language processing	https://arxiv.org/pdf/2302.08575	Foundation Models in Natural Language Processing: A Survey	Foundation models, such as BERT and GPT, have transformed the NLP landscape by utilizing extensive datasets and advanced architectures that enable high performance and flexibility across diverse tasks. They leverage unsupervised pre-training on large corpora to autonomously learn	-
0872278-c384--	What are the major challenges in instruction tuning for large language	https://arxiv.org/pdf/2308.10792	Instruction Tuning for Large Language Models: A Survey	The paper identifies five major challenges in instruction tuning for large language models: 1. **Instruction Quality** - Crafting high-quality, diverse instructions is complex, and current datasets lack variety. 2. **Dataset Limitations** - SFT models perform well on tasks represented in training	-

CometML Opik

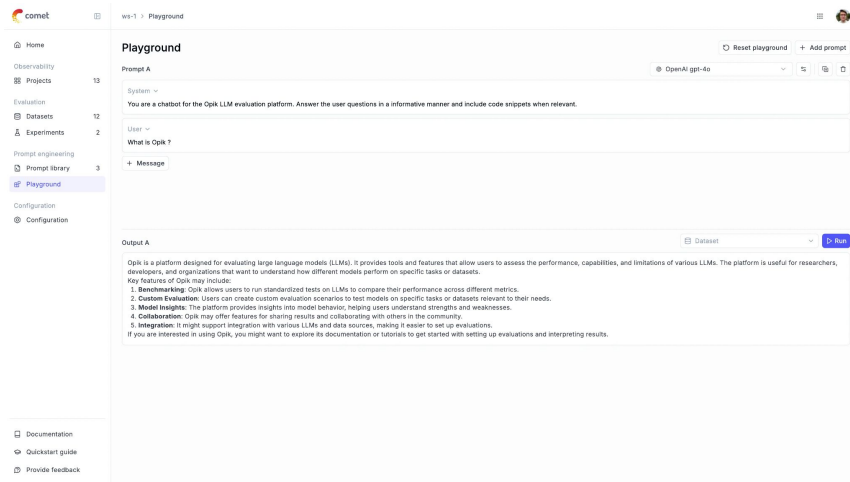


The image shows the LangSmith interface for an experiment named 'gpt-4o-mini-with-reflection-step-46535407'. The top navigation bar includes 'Home', 'Datasets & Experiments', and 'gpt-4o-mini-with-reflection-st...'. The 'Runs' section is active, showing a list of runs with columns for 'Error', 'Latency', 'Cost', 'Metadata', 'Feedback', and 'Reference Example'. The 'Runs' table shows a single run with a score of 0.80. Below the table, there is a 'Filter' button and a 'Compare' button. The main content area displays a table of run results with columns for 'Error', 'Latency', 'Cost', 'Metadata', 'Feedback', and 'Reference Example'. The table lists several runs, including 'revision_id: 807f8ed-dirty', 'revision_id: 807f8ed-dirty', 'revision_id: 807f8ed-dirty', 'revision_id: 807f8ed-dirty', 'revision_id: 807f8ed-dirty', 'revision_id: 807f8ed-dirty', 'revision_id: 807f8ed-dirty', 'revision_id: 807f8ed-dirty', and 'revision_id: 807f8ed-dirty'. The table also includes a 'Feedback' column with a 'score_string: accuracy' sub-column.

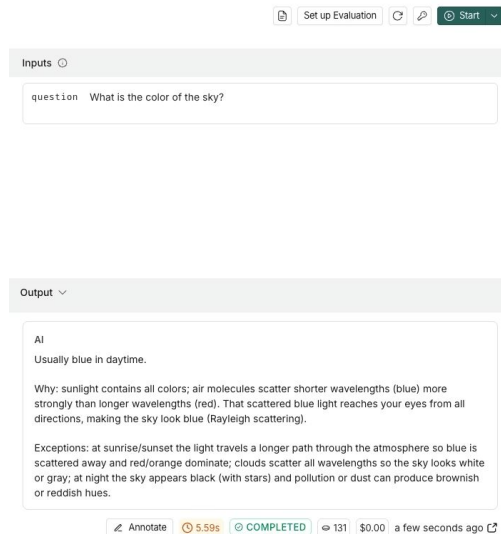
Error	Latency	Cost	Metadata	Feedback	Reference Example
82.02s	\$0.0031017	revision_id: 807f8ed-dirty	score_string: accuracy 0.80	#9885 @ chat-langchain	
52.01s	\$0.0031017	revision_id: 807f8ed-dirty	score_string: accuracy 1.00	#9885 @ chat-langchain	
53.98s	\$0.00270865	revision_id: 807f8ed-dirty	score_string: accuracy 1.00	#9885 @ chat-langchain	
49.98s	\$0.00229935	revision_id: 807f8ed-dirty	score_string: accuracy 1.00	#9885 @ chat-langchain	
51.32s	\$0.00282495	revision_id: 807f8ed-dirty	score_string: accuracy 1.00	#9885 @ chat-langchain	
50.42s	\$0.00177045	revision_id: 807f8ed-dirty	score_string: accuracy 1.00	#9885 @ chat-langchain	
50.54s	\$0.002872	revision_id: 807f8ed-dirty	score_string: accuracy 1.00	#9885 @ chat-langchain	
45.45s	\$0.00213895	revision_id: 807f8ed-dirty	score_string: accuracy 1.00	#9885 @ chat-langchain	
51.38s	\$0.0028935	revision_id: 807f8ed-dirty	score_string: accuracy 0.80	#9885 @ chat-langchain	
48.21s	\$0.00233625	revision_id: 807f8ed-dirty	score_string: accuracy 1.00	#9885 @ chat-langchain	
48.73s	\$0.0029038	revision_id: 807f8ed-dirty	score_string: accuracy 0.80	#9885 @ chat-langchain	

LangSmith

AgentOps platforms. Prompt playgrounds

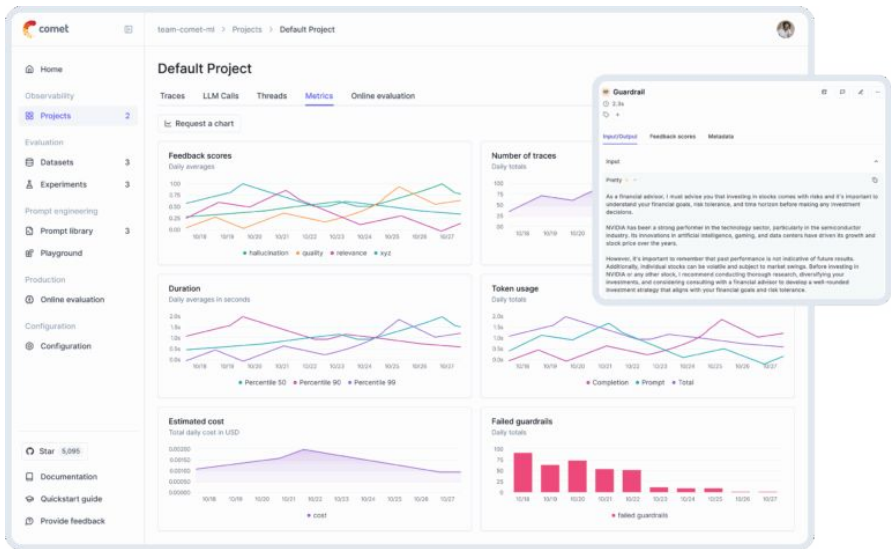


CometML Opik

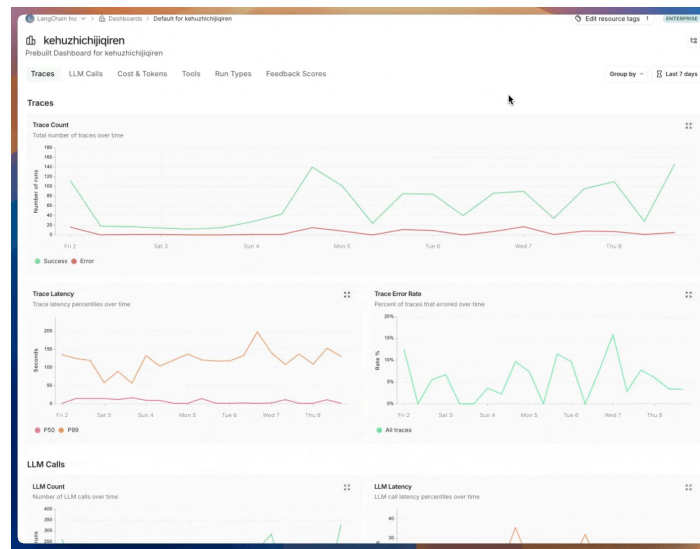


LangSmith

AgentOps platforms. Monitoring



CometML Opik



LangSmith

Current & Promising Applications

- **Software Development:** Autonomous coding agents that can write, debug, and test code.
 - Cursor
 - Copilot
 - Devin
- **Business Automation:** Agents that can handle customer support inquiries, schedule meetings, and analyze sales data.
 - [Fin.AI](#)
 - Custom solutions (Yandex, Avito, e.t.c)
- **Personal Productivity:** A "do-everything" personal assistant.
 - All kinds of custom n8n workflows
 - Cleo for financial tasks
 - OpenAI GPTs
 - FitBit AI for health
- **Scientific Research:** Agents that can read papers, run simulations, and analyze experimental data.
 - AlphaXiv
 - Paper2Agent [arXiv:2509.06917 \[cs.AI\]](#)
- **Gaming & Entertainment:** Creating dynamic, intelligent Non-Player Characters (NPCs).
 - Inworld AI
 - Convai
 - Replika

Insights from enterprise. Anthropic on deep research

Overview

- **Architecture:** One coordinator + multiple specialized worker agents operating in parallel, each with its own context.
- **Strengths:** Handles open-ended tasks where step-by-step workflows cannot be predefined.
- **Costs:** ~15× higher token usage compared to single-agent chats → serious cost-efficiency considerations.
- **Production Challenges:**
 - Stateful agents accumulate errors → require checkpoints and safe recovery.
 - Deploying updates is tricky because agents may be mid-task.
 - Debugging is harder due to non-linear reasoning and multiple interacting agents; requires strong observability.

- **Best Practices:**

- Provide clear roles, goals, and output formats for worker agents.
- Use parallelism carefully to balance speed with coordination complexity.
- Evaluate by final output quality rather than internal step logic.

Concerns and Ethical Considerations

- **Security:** If an agent can call APIs, it can potentially cause damage. How do we secure it?
 - Solutions: Human-in-the-loop, Guardrails
- **Reliability & Alignment:** How do we ensure the agent stays on task and doesn't pursue unintended goals?
 - **The Paperclip Maximizer:** A classic thought experiment about an AI given a simple goal (make paperclips) that consumes all of Earth's resources to achieve it, highlighting the danger of misaligned objectives.
- **Data Privacy:** Agents connected to personal data (emails, files) pose significant privacy risks.
- **Scalability and Cost:** Running complex agent loops can be computationally expensive.
 - Solution: Small language models are the future of Agentic AI. [arXiv:2506.02153 \[cs.AI\]](#)

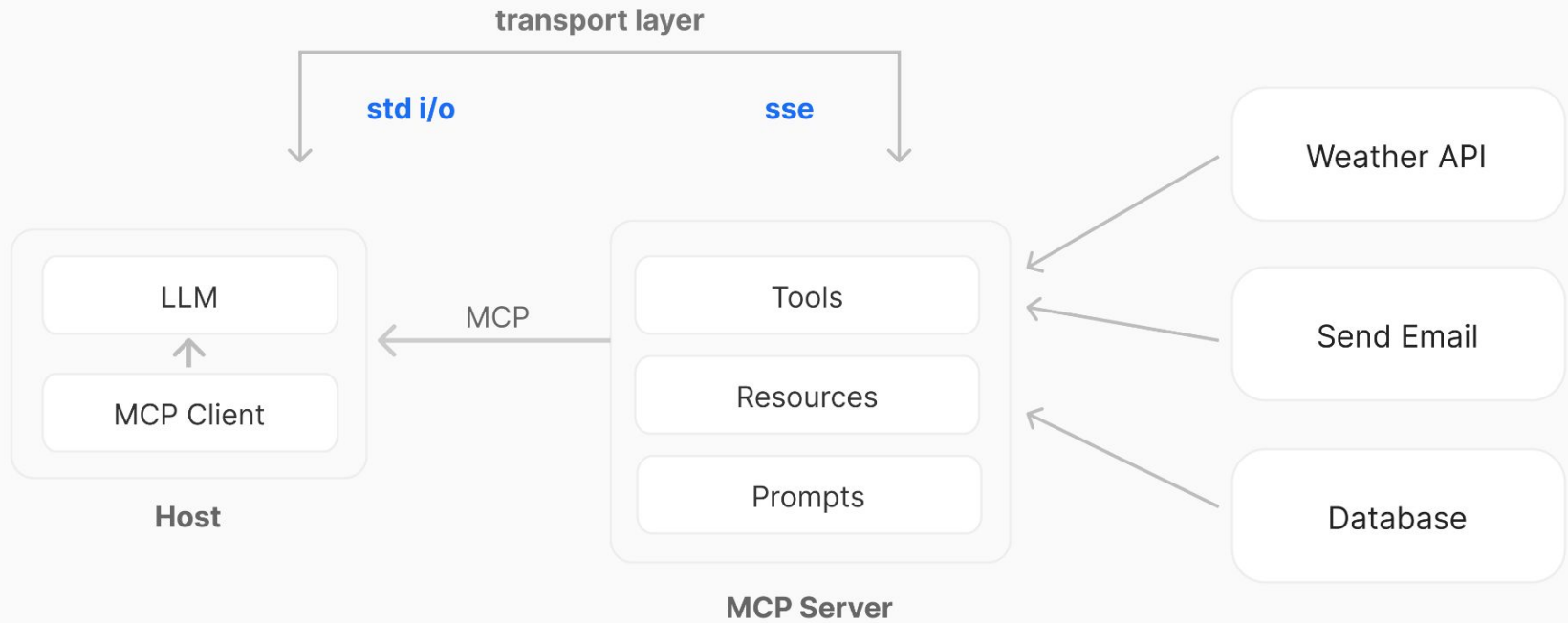
Conclusion & Q&A:

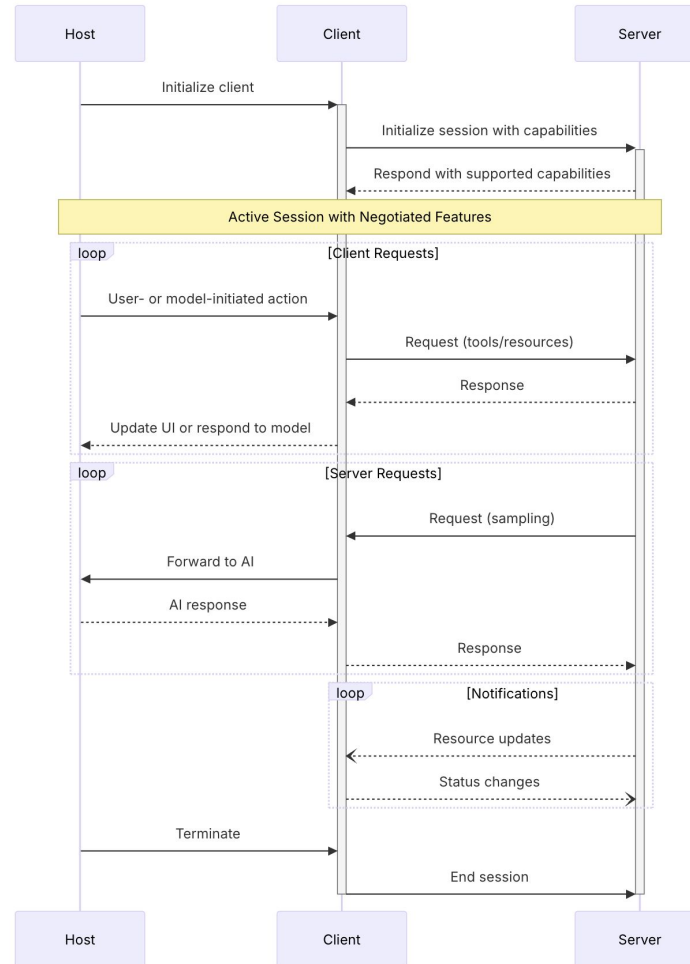
LLM Agents represent a fundamental shift from passive text generators to active participants in our digital world. While the technology is still nascent, its potential to revolutionize how we interact with computers and automate complex tasks is immense.

Appendix.

MCP under the hood

MCP Client - Server





How communication is performed in MCP

JSON RPC messages over SSE/stdio/Streamable HTTP. Typical Request schema:

Requests

```
{
  jsonrpc: "2.0",
  id: number | string,
  method: string,
  params?: object
}
```

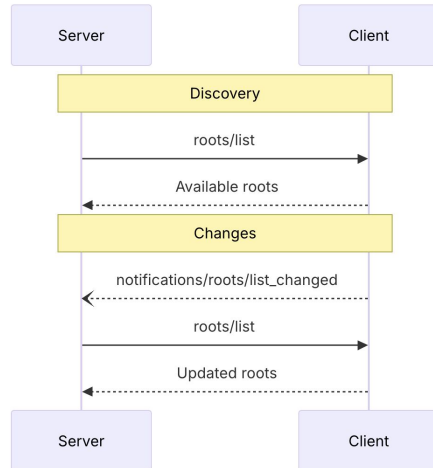
Responses

```
{
  jsonrpc: "2.0",
  id: number | string,
  result?: object,
  error?: {
    code: number,
    message: string,
    data?: unknown
  }
}
```

Client features. Roots

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "roots/list"
}
```

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "roots": [
      {
        "uri": "file:///home/user/projects/myproject",
        "name": "My Project"
      }
    ]
  }
}
```



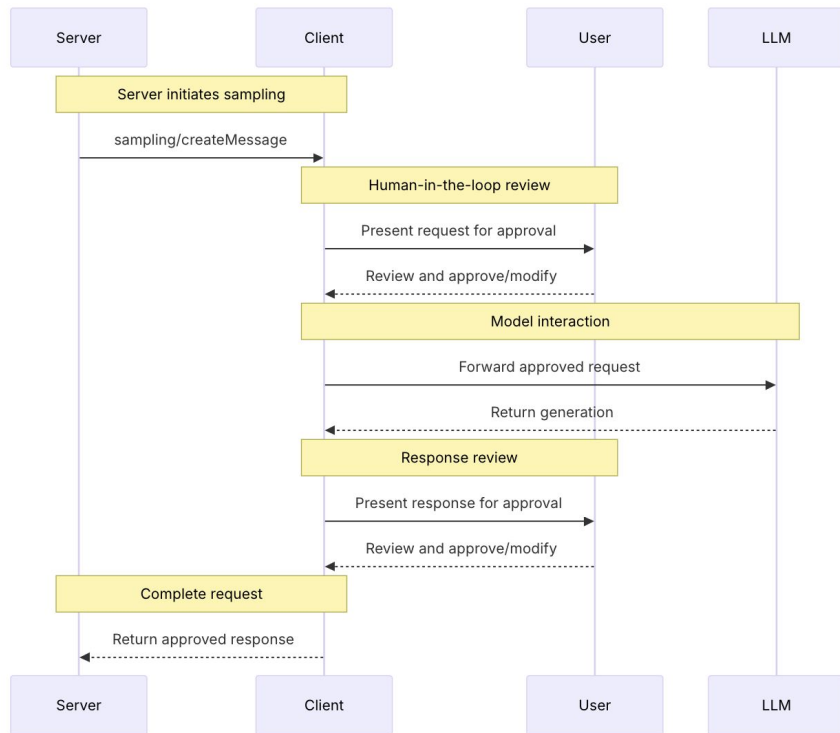
Client Features. Sampling

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "sampling/createMessage",
  "params": {
    "messages": [
      {
        "role": "user",
        "content": {
          "type": "text",
          "text": "What is the capital of France?"
        }
      }
    ],
    "modelPreferences": {
      "hints": [
        {
          "name": "claude-3-sonnet"
        }
      ]
    },
    "intelligencePriority": 0.8,
    "speedPriority": 0.5
  },
  "systemPrompt": "You are a helpful assistant.",
  "maxTokens": 100
}
```



```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "role": "assistant",
    "content": {
      "type": "text",
      "text": "The capital of France is Paris."
    },
    "model": "claude-3-sonnet-20240307",
    "stopReason": "endTurn"
  }
}
```

Sampling. Message flow



Server features

Primitive	Control	Description	Example
Prompts	User-controlled	Interactive templates invoked by user choice	Slash commands, menu options
Resources	Application-controlled	Contextual data attached and managed by the client	File contents, git history
Tools	Model-controlled	Functions exposed to the LLM to take actions	API POST requests, file writing

Server features. Prompts

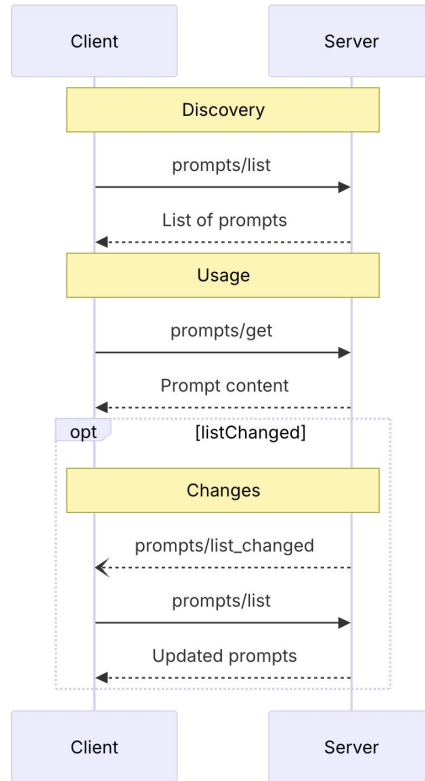
```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "prompts/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

```
"id": 1,
"result": {
  "prompts": [
    {
      "name": "code_review",
      "description": "Asks the LLM to analyze code quality and suggest",
      "arguments": [
        {
          "name": "code",
          "description": "The code to review",
          "required": true
        }
      ]
    }
  ]
},
"nextCursor": "next-page-cursor"
}
```

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "prompts/get",
  "params": {
    "name": "code_review",
    "arguments": {
      "code": "def hello():\n    print('world')"
    }
  }
}
```

```
"id": 2,
"result": {
  "description": "Code review prompt",
  "messages": [
    {
      "role": "user",
      "content": {
        "type": "text",
        "text": "Please review this Python code:\ndef hello():\n"
      }
    }
  ]
}
```

Server features. Prompts. Message Flow



Server features. Resources

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "resources/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

```
"id": 1,
"result": {
  "resources": [
    {
      "uri": "file:///project/src/main.rs",
      "name": "main.rs",
      "description": "Primary application entry point",
      "mimeType": "text/x-rust"
    }
  ],
  "nextCursor": "next-page-cursor"
}
```

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "resources/read",
  "params": {
    "uri": "file:///project/src/main.rs"
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "contents": [
      {
        "uri": "file:///project/src/main.rs",
        "mimeType": "text/x-rust",
        "text": "fn main() {\n    println!(\"Hello wo"
      }
    ]
  }
}
```

Server features. Resources Templates

Request:

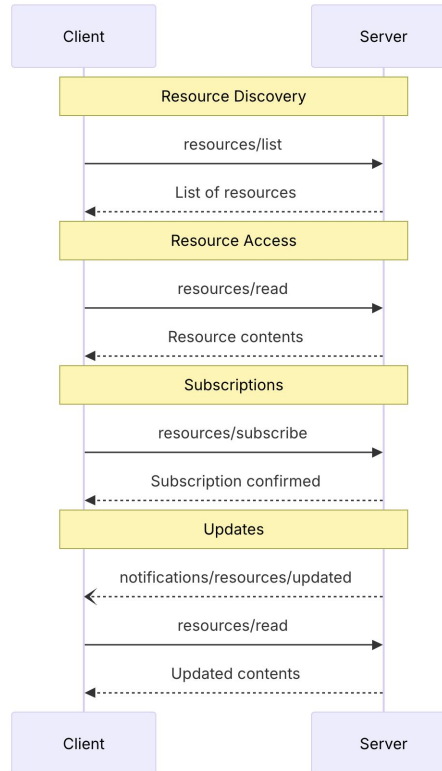
```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "resources/templates/list"
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "resourceTemplates": [
      {
        "uriTemplate": "file:///path",
        "name": "Project Files",
        "description": "Access files in the project directory",
        "mimeType": "application/octet-stream"
      }
    ]
  }
}
```

Resources Message Flow

Message Flow



Server features. TOOLS

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {
    "cursor": "optional-cursor-value"
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "get_weather",
        "description": "Get current weather information for a location",
        "inputSchema": {
          "type": "object",
          "properties": {
            "location": {
              "type": "string",
              "description": "City name or zip code"
            }
          },
          "required": ["location"]
        }
      },
      {
        "name": "get_weather",
        "description": "Get current weather information for a location",
        "inputSchema": {
          "type": "object",
          "properties": {
            "location": {
              "type": "string",
              "description": "City name or zip code"
            }
          },
          "required": ["location"]
        }
      }
    ],
    "nextCursor": "next-page-cursor"
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "New York"
    }
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Current weather in New York:\nTemperature: 72°F\nCondition: Partly Cloudy"
      }
    ],
    "isError": false
  }
}
```

Tools Message Flow

